



**Abertay
University**

Minecraft Intrusion Detection and Notification System

An intrusion detection system monitoring a virtual Minecraft house connected to an RPi to send intrusion alerts via discord and web application dashboard.

Oliver Wright

CMP408: IoT and Cloud Secure Development

2025/26

Note that Information contained in this document is for educational purposes.

Contents

1	Introduction	1
1.1	Background.....	1
1.2	Objectives	1
2	Development.....	2
2.1	Hardware Setup.....	2
2.2	Software Development.....	2
2.2.1	Minecraft Scripts (Lua)	2
2.2.2	RPi Scripts (Python, C).....	2
2.2.3	Web Application (HTML, CSS, JS, Python).....	4
2.3	Cloud Development.....	4
3	Conclusion.....	5
4	References	6
	Appendices.....	7
	Appendix A – RPi Scripts	7
4.1.1	IntrusionMonitor.c	7
4.1.2	Makefile.....	8
4.1.3	Main.py	9
4.1.4	alertServer.py.....	9
4.1.5	flipLed.py	11
4.1.6	database.py	12
4.1.7	notifications.py.....	13
4.1.8	config.py	14
	Appendix B – Web Application Scripts.....	15
4.1.9	index.html	15
4.1.10	style.css.....	16
4.1.11	script.js.....	17
4.1.12	lambda_function.py.....	19
	Appendix C – Images/Screenshots.....	20

1 INTRODUCTION

1.1 BACKGROUND

IoT security has always been a struggle due to its production being unable to keep up with its rapid growth in demand. “As the number of connected IoT devices continually increase, security issues are exponentially multiplied [...]. Moreover, traditional security mechanisms cannot be directly implemented to IoT technologies due to their designed system i.e. limited power as well as these large number of connected devices raise heterogeneity and scalability issues”, (Hanan, Shafiq, & Irfan, 2019). As a result, IoT devices are rarely fully secure, making secure development an essential current necessity.

One huge advantage of IoT is the ability to stay connected to a device over a distance which is a vital feature of this project. In this project, an intrusion detection system was developed using a Raspberry Pi (RPI) with alert notifications being sent to the device owner. The project also uses external components and cloud functionality to notify the device owner. The unique aspect of this project is the fact that the environment being monitored is a digital one within the video game Minecraft using a mod called “CC:Tweaked”. While this system currently monitors a virtual environment, the backend could easily be modified for real world use.

There are many similar projects that prove the viability of using a Raspberry Pi as a strong platform of choice for an IoT Intrusion Detection System. “Development of surveillance robot based on face recognition using Raspberry-PI and IOT” published in Microprocessors and Microsystems volume 96, (Meddeb, Abdellaoui, & Houaidi, 2023) is a great example of this. Another strong example is “Zeek Intrusion Detection on Raspberry Pi for IoT-based Agriculture Monitoring Systems: Preliminary Investigation”, (Huda, Musthafa, & Nogami, 2024).

1.2 OBJECTIVES

- Develop an intrusion detection system using an RPi monitoring a virtual environment (Minecraft).
- Include RPi component peripherals in the alert process to effectively notify the user.
- Feature cloud functionality in the form of a database storing logs and a web application displaying logs.
- Include LKM functionality to monitor the GPIO pins and count the number of alerts using this.
- Follow proper secure practices to prevent vulnerabilities where possible.

2 DEVELOPMENT

2.1 HARDWARE SETUP

This project contains two physical hardware components attached to the RPi. These two components are an LED light and a Buzzer. These two components are connected to the RPi using a breadboard and are both triggered when an intrusion alert is sent to the flask server (*figure 1*).

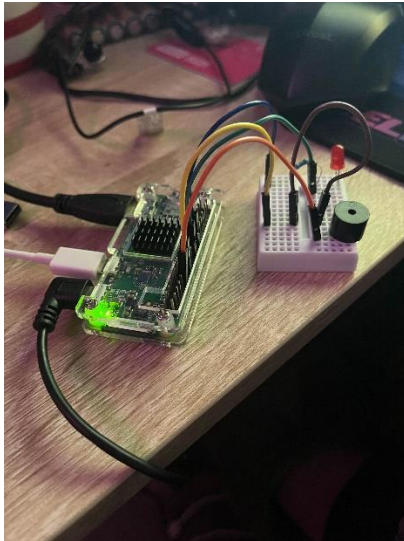


Figure 1 Picture of RPi setup

2.2 SOFTWARE DEVELOPMENT

2.2.1 Minecraft Scripts (Lua)

pingRPi.lua (*figure 2[C]*)

This is the script written and hosted on the in-game Minecraft terminal and acts as a sensor detecting an intruder. When run, the script sends a POST request to the flask HTTP server alerting that there is an intruder. The POST request contains the flask server address, a message ("Alarm Triggered: Intruder") and the API key required to make requests to the flask server. After sending the request, the Minecraft terminal waits for a response. If a response is received, then the script will finish, however, if no response is received then the error will be printed before finishing the script. This part of the project made use of the CC:Tweaked API which can be found in this paper's references (SquidDev, 2025).

2.2.2 RPi Scripts (Python, C)

intrusionMonitor.c (4.1.1 [A])

This is an LKM created to monitor the GPIO pins containing the LED from kernel space. The module counts LED activations and sends the total number to user space. A Makefile script (*figure 4 [C]*) was

created to make a “.ko” file of intrusionMonitor.c (*figure 6 [C]*). This file was then run as a service and can be seen updating when the “echo” command is used (*figure 7 [C]*).

Config.py, .env (4.1.8 [A], *figure 8 [C]*)

Seeing as the project has quite a few sensitive and highly configurable variables, a config file was created. This file is used to hold the configurable options and call sensitive environment variables from the “.env” environment file. The “.env” file was set to read only and added to “.gitignore”.

alertServer.py (4.1.4 [A])

A script is used to launch a basic Flask HTTP server. This allows the connection between the in-game terminal and the main program running on the RPi. To ensure alerts only come from verified sources an API key was generated using the following script (*figure 4 [C]*). Storing this API key in the script would also be a security flaw, therefore, it was set as an environment variable in the “.env” file. Every request sent to the HTTP server must now contain the API key and is checked by the verifyKey function.

flipLed.py (4.1.5 [A])

The breadboard components, LED and buzzer, are managed within this script. It starts with a function “GPIOController” to initialize the GPIO pins for the LED’s and buzzer’s circuits. Next the flipLED and flipBuzz functions are called which turn on and off the LED and buzzer in a beeping pattern. Both the LED and buzzer and a function to manually turn them off, this was mostly for testing purposes. Lastly, a cleanup function cleans the GPIO pins configuration and ensures the components do not run when they are not supposed to. This function is called in “main.py”.

Database.py (4.1.6 [A])

The table name “alertLogs” is set up as a variable and a session is started to connect to it. Next the “logAlert” function is called. The sanitizeMessage function is called to prevent allowing SQL injection into the database. A new item field “ID” is created and the uuid library is used to generate a new ID. Next, the function attempts to input a new item into the table, taking in an ID, message and a timestamp as item fields. Errors are handled using python’s built in “try”, “except” feature.

Notifications.py (4.1.7 [A])

Alert notifications are handled here. The sendDiscord function sends an alert in real-time to a Discord server when an intrusion is detected. It does this using Discords built in webhook feature which can be requested for free. This can be found in server settings > integrations > webhooks. After that the message is constructed and a post request is sent. If this fails an error is displayed.

Main.py (4.1.3 [A])

This initializes the system and calls functions from other files. First the GPIOcontroller and setController functions are called to initialize the GPIO configuration for this project. Next, the HTTP alert server is run. Lastly, the cleanup function makes use of the imported “atexit” library to call the cleanup function. This simply runs the cleanup function upon exiting the program.

2.2.3 Web Application (HTML, CSS, JS, Python)

Index.html, style.css (4.1.9 [B], 4.1.10 [B])

The web application “Intrusion Dashboard” (figure 9 [C]) is separated into 3 files, “index.html”, “style.css” and “script.js”. The first two are basic HTML and CSS to create a respectable looking dashboard UI for the user to view and interact with.

Script.js (4.1.11 [B])

Logs are pulled from the database using an API request and come in the form of a JSON file. The logs are then imbedded into the HTML to display them on the web application. If no logs are available, then a placeholder message is shown. The logs can be manually refreshed using the refresh button or by waiting for the 5 second update interval.

Lambda_function.py (4.1.12 [B])

The lambda function was required to connect the database to the web application. The function starts with CORS header to allow the web application to access the database. Next the data is placed into a list, and the list is reversed to show the newest logs first. The list is then cut off at 20 alerts as old alerts do not necessarily need to be shown on the intrusion dashboard. If the function is successful, then the alert logs are returned inside a JSON dump for the web application. If the function is unsuccessful then errors are returned in a JSON dump instead.

2.3 CLOUD DEVELOPMENT

AWS services – Amplify, S3, Lambda, API Gateway, DynamoDB

Amazon Web Services (AWS) was used for the cloud functionality of this project. There are two main cloud components: a database to hold intrusion alert logs, and a web application security dashboard to display those logs. To achieve this, DynamoDB was used for the database. A table named “alertLogs” was created, (figure 3 [C]), which stores an ID (primary key), message (e.g. “Alarm Triggered: Intruder Detected”) and a timestamp of when the alert was triggered.

Getting a static web application proved to be much more complicated than the database setup. Originally S3 was going to be used, however, due to permission complications on the AWS account, Amplify was used instead. Previously, an S3 bucket was created with the web application files, so this bucket was selected as the option for Amplify to host as a static web application. Lambda and API Gateway were used to connect the database to the web application.

3 CONCLUSION

An intrusion detection system was successfully developed making use of a variety of programming languages, concepts and cloud features. Upon sending an alert from the in-game virtual environment, component peripherals are triggered, a notification is sent, a database is updated and a web application displays these changes in real time. A basic LKM is used to monitor the hardware, and secure practices were deployed throughout the entire project.

A variety of security techniques were deployed upon the project's development process. Error handling is present in almost every file. This is to prevent the program from crashing if an error does occur and instead provides insight into the issue. Environment variables were set to hide sensitive information such as API keys to prevent them from being viewable in the source code. Input sanitization was used when making requests to the database to stop SQL injection attempts. All preset objectives of the project were effectively met concluding its development.

4 REFERENCES

- Hanan, A., Shafiq, U. R., & Irfan, U. (2019, July). *Security in Internet of Things: Issues, Challenges and Solutions*. Retrieved from ResearchGate: https://www.researchgate.net/publication/326579980_Security_in_Internet_of_Things_Issues_Challenges_and_Solutions
- Huda, S., Musthafa, M. B., & Nogami, Y. (2024, December 17). *Zeek Intrusion Detection on Raspberry Pi for*. Retrieved from ieeexplore: <https://ieeexplore-ieee-org.libproxy.abertay.ac.uk/stamp/stamp.jsp?tp=&arnumber=10791229>
- Meddeb, H., Abdellaoui, Z., & Houaidi, F. (2023, February). *Development of surveillance robot based on face recognition using Raspberry-Pi and IOT*. Retrieved from ScienceDirect: <https://www-sciencedirect-com.libproxy.abertay.ac.uk/science/article/pii/S0141933122002575>
- SquidDev. (2025). *CC:Tweaked*. Retrieved from CC:Tweaked: <https://tweaked.cc/>

APPENDIX A – RPI SCRIPTS

4.1.1 IntrusionMonitor.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "alertCounter"
#define MAJOR_NUM 20

//global counter var
static int alertCount = 0;

static int alertOpen(struct inode *inode, struct file *file) {
    pr_info("Intrusion Monitor: %s\n", __func__);
    return 0;
}

static int alertRelease(struct inode *inode, struct file *file) {
    pr_info("Intrusion Monitor: %s\n", __func__);
    return 0;
}

static ssize_t alertRead(struct file *file, char *buffer, size_t
length, loff_t *offset) {
    char temp[16];
    int len;

    pr_info("Intrusion Monitor: %s %u\n", __func__, length);

    if (*offset > 0)
        return 0;

    len = sprintf(temp, "%d\n", alertCount);

    if (copy_to_user(buffer, temp, len))
        return -EFAULT;

    *offset = len;
    return len;
}

static ssize_t alertWrite(struct file *file, const char *buffer,
size_t length, loff_t *offset) {
```

```

    pr_info("Intrusion Monitor: %s %u\n", __func__, length);

    alertCount++;
    printk("Intrusion Monitor // Alert Counter: %d\n", alertCount);

    return length;
}

struct file_operations alertFops = {
    .owner = THIS_MODULE,
    .open = alertOpen,
    .release = alertRelease,
    .read = alertRead,
    .write = alertWrite,
};

int __init alertInit(void) {
    int ret ;
    pr_info("Intrusion Monitor: %s\n", __func__);

    ret = register_chrdev(MAJOR_NUM, DEVICE_NAME, &alertFops);
    if (ret != 0)
        return ret;

    printk("Intrusion Monitor: alertCounter loaded \n");
    return 0;
}

void __exit alertExit(void) {
    unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
    printk("Intrusion Monitor: alertCounter unloaded, Final Count:
%d\n", alertCount);
}

module_init(alertInit);
module_exit(alertExit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Oliver Wright");
MODULE_DESCRIPTION("Alert Counter LKM");

=====End File=====

```

4.1.2 Makefile

```

obj-m += intrusionMonitor.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD)
modules

```

```
clean:
    make -C /lib/modules/$(shell uname -r)/ build M=$(PWD) clean

=====End File=====
```

4.1.3 Main.py

```
import alertServer
import flipLed
import atexit
from config import host, port

def main():
    #while True:
    print("Intrusion detection system starting...")

    flipLed.GPIOController()

    alertServer.setController(flipLed)

    atexit.register(flipLed.cleanup)

    alertServer.startServer(host, port)

main()

=====End File=====
```

4.1.4 alertServer.py

```
from flask import Flask, request
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from datetime import datetime
from config import dataDivider, MINECRAFT_API_KEY
from notifications import sendDiscord, sendEmail
from database import logAlert

gpio = None

app = Flask(__name__)
limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=["100 per day"],
)

def setController(gpioModule):
    global gpio
```

```

        gpio = gpioModule
        print("gpio functions connected")

def verifyKey():
    key = request.headers.get('API_KEY')
    if not key or key != MINECRAFT_API_KEY:
        return "API key error"
    return None

def logLKM():
    try:
        with open('/dev/alertCounter', 'w') as f:
            f.write('1')
            print("Logged to kernel")
    except Exception as e:
        print(f"Kernel module error: {e}")

def getLKMCount():
    try:
        with open('/dev/alertCounter', 'r') as f:
            return int(f.read().strip())
    except:
        return 0

@app.route('/alert', methods=['GET', 'POST'])
@limiter.limit("5 per minute")
def handleAlert():
    keyError = verifyKey()
    if keyError:
        print("Invalid API key")
        return False

    timestamp = datetime.now()
    rawData = request.get_data(as_text=True).strip()
    print("RawData: ", rawData)

    print(dataDivider)
    print(f"[{timestamp}] ALERT FROM MC")
    print(f>Data: {rawData}")
    print(dataDivider)

    logAlert(f"{rawData}", timestamp)
    sendDiscord(f"{rawData}, [{timestamp}]")
    logLKM()

    if gpio:
        gpio.flipLED()
        #gpio.flipBuzz()
    else:
        print("GPIO not connected")

```

```

        return "Success", 200

def startServer(host, port):
    print("Server running on port 8000")
    app.run(host, port)

=====End File=====

```

4.1.5 flipLed.py

```

import RPi.GPIO as GPIO
import time
import threading
from config import LED_PIN, LED_DURATION, BUZZ_PIN, BUZZ_DURATION

initStatus = False
LEDTimer = None
buzzTimer = None

def GPIOController():
    global initStatus

    if initStatus:
        return

    GPIO.setmode(GPIO.BCM)
    GPIO.setup(LED_PIN, GPIO.OUT)
    GPIO.output(LED_PIN, GPIO.LOW)
    GPIO.setup(BUZZ_PIN, GPIO.OUT)
    GPIO.output(BUZZ_PIN, GPIO.LOW)

    initStatus = True

def flipLED():
    global LEDTimer
    if not initStatus:
        print("Not initialized, quitting...")
        return

    if LEDTimer:
        LEDTimer.cancel()

    GPIO.output(LED_PIN, GPIO.HIGH)
    print("LED on")

    LEDTimer = threading.Timer(LED_DURATION, cutLED)
    LEDTimer.start()

def flipBuzz():
    global buzzTimer

```

```

buzzBeeps = 3
if not initStatus:
    print("Not initialized, quitting...")
    return

if buzzTimer:
    buzzTimer.cancel

print("Buzzer on")
for i in range(buzzBeeps):
    GPIO.output(BUZZ_PIN, GPIO.HIGH)
    time.sleep(1)
    cutBuzz()
    time.sleep(1)

def cutLED():
    GPIO.output(LED_PIN, GPIO.LOW)
    print("LED off")

def cutBuzz():
    GPIO.output(BUZZ_PIN, GPIO.LOW)
    print("Buzzer off")

def cleanup():
    GPIO.cleanup()
    print("GPIO cleaned")

```

4.1.6 database.py

```

import boto3
import os #<- for env vars
from datetime import datetime
import uuid
import re
from config import AWS_REGION, AWS_ACCESS_KEY, AWS_SECRET_KEY,
AWS_SESSION_TOKEN

tableName = "alertLogs"

session = boto3.Session(
    region_name = AWS_REGION,
    aws_access_key_id = AWS_ACCESS_KEY,
    aws_secret_access_key = AWS_SECRET_KEY,
    aws_session_token = AWS_SESSION_TOKEN
)

dynamodb = session.resource('dynamodb')

```

```

table = dynamodb.Table(tableName)

def testDB():
    timestamp = datetime.now()
    logAlert("Test", timestamp)
    print("Done")

def sanitizeMessage(message):
    if len(message) > 70:
        message = message[:70]

    newMessage = ""
    bannedChars = [".", "!", "?", "-"]
    for char in message:
        if char not in bannedChars:
            newMessage += char

    return newMessage

def logAlert(message, timestamp):
    try:
        if not message or not timestamp:
            print("Invalid Data")
            return False

        message = sanitizeMessage(message)
        itemID = str(uuid.uuid4())

        item = {
            'id': itemID,
            'timestamp': timestamp.isoformat(),
            'message': message
        }

        table.put_item(Item=item)
        print("logged alert db")
        return True

    except Exception as e:
        print(f"DB Error: {e}")
        return False

=====End File=====

```

4.1.7 notifications.py

```

import requests
from config import appEmail, appPass, discordURL

```

```

url = discordURL

def sendDiscord(message):
    global url

    data = {
        "content" : message,
        "username" : "Soup Alert",
    }

    response = requests.post(url, json = data)

    try:
        response.raise_for_status()
    except requests.exceptions.HTTPError as err:
        print(err)
    else:
        print(f"Payload delivered, {response.status_code}")

def sendEmail(userEmail):
    print("TBC")

=====End File=====

```

4.1.8 config.py

```

from dotenv import load_dotenv
import os

load_dotenv()

dataDivider = "===== "

host = "0.0.0.0"
port = "8000"
MINECRAFT_API_KEY = os.getenv("MINECRAFT_API_KEY")

AWS_REGION = os.getenv("AWS_REGION")
AWS_ACCESS_KEY = os.getenv("AWS_ACCESS_KEY")
AWS_SECRET_KEY = os.getenv("AWS_SECRET_KEY")
AWS_SESSION_TOKEN = os.getenv("AWS_SESSION_TOKEN")

appEmail = None
appPass = None

#discord webhook
discordURL = os.getenv("discordURL")

#GPIO info too!
LED_PIN = 23
LED_DURATION = 5

```

```
BUZZ_PIN = 7
BUZZ_DURATION = 3

=====End File=====
```

APPENDIX B – WEB APPLICATION SCRIPTS

4.1.9 index.html

```
<!DOCTYPE HTML>
<html>
  <head lang="en">
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.
min.css" rel="stylesheet">
    <title>Intrusion Dashboard</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>

    <div class="header">
      <h2>Minecraft Intrusion Detection System</h2>
    </div>
    <!-- alerts-->
    <div class="dashboardText">
      <h2>Security Dashboard</h2>
    </div>
    <div class="alertsContainer">
      <div class="card">
        <div class="cardHeader">
          <h3>Recent Alerts</h3>
          <button id="refreshButton">Refresh</button>
        </div>
        <div class="cardBody">
          <div id="alertList">
            <p class="noAlerts">No alerts to display</p>
          </div>
        </div>
      </div>
    </div>

    <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bu
ndle.min.js"></script>
    <script src="script.js"></script>
```

```
        </body>

</html>

=====End File=====
```

4.1.10 style.css

```
body {
    min-height: 100vh;
    font-family: 'Trebuchet MS'
}

.header {
    background-color: darkslategrey;
    color: white;
    text-align: center;
    margin-bottom: 20px;
    padding: 20px;
}

.dashboardText {
    text-align: center;
    margin-bottom: 30px;
}

.dashboardText h2 {
    color: darkslategrey;
}

.alertsContainer {
    margin: auto;
    padding: 20px;
}

.card {
    background: white;
    border: 2px solid darkslategrey;
    border-radius: 0px;
}

.cardHeader {
    background: darkslategray;
    color: white;
    padding: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.cardBody {
    padding: 20px;
```

```

}

#refreshButton {
  background: lightgrey;
  color: darkslategrey;
  padding: 10px;
  cursor: pointer;
  font-size: 14px;
}

#refreshButton:hover {
  background: goldenrod;
}

.alertItem {
  border-left: 4px solid darkslategrey;
  background: whitesmoke;
  padding: 15px;
  margin-bottom: 10px;
}

.alertItem strong {
  color: darkred;
}

.alertItem small {
  color: gray;
}

.noAlerts {
  text-align: center;
  color: gray;
  margin: 20px 0;
}

=====End File=====

```

4.1.11 script.js

```

const API_URL = "https://o6aofua11m.execute-api.us-east-1.amazonaws.com/default/getAlertLogs"

function loadAlerts() {
  fetch(API_URL)
    .then(response => response.json())
    .then(data => {
      const alertList = document.getElementById('alertList');

      if (!data.logs || data.logs.length === 0) {
        alertList.innerHTML = '<p class="noAlerts">No
alerts</p>';

```

```

        return;
    }

    displayAlerts(data.logs);

})

.catch(error => {
    console.log('Error: ', error);
})

}

function displayAlerts(logs) {
    const alertList = document.getElementById('alertList');

    let html = '';
    for (let log of logs) {
        html += `
        <div class="alertItem">
            <strong>${log.message}</strong><br>
            <small>${formatTime(log.timestamp)}</small>
        </div>
        `;
    }

    alertList.innerHTML = html;
}

function formatTime(timestamp) {
    const alertTime = new Date(timestamp);
    const now = new Date();
    const diff = Math.floor((now - alertTime) / 1000);

    return alertTime.toLocaleString();
}

document.getElementById('refreshButton').addEventListener('click',
loadAlerts);
loadAlerts();
setInterval(loadAlerts, 5000);

=====End File=====

```

4.1.12 lambda_function.py

```
import json
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('alertLogs')

def lambda_handler(event, context):
    headers = {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type',
        'Access-Control-Allow-Methods': 'GET,OPTIONS',
        'Content-Type': 'application/json'
    }

    if event.get('httpMethod') == 'OPTIONS':
        return {
            'statusCode': 200,
            'headers': headers,
            'body': ''
        }

    try:

        response = table.scan()
        items = response['Items']

        items.sort(key=lambda x: x['timestamp'], reverse=True)
        items = items[:20]

        return {
            'statusCode': 200,
            'headers': headers,
            'body': json.dumps({'logs': items})
        }

    except Exception as e:
        return {
            'statusCode': 500,
            'headers': headers,
            'body': json.dumps({'error': str(e)})
        }

=====End File=====
```

APPENDIX C – IMAGES/SCREENSHOTS

```
local API_KEY = "k9v1u9cPIyTtWI1UMD1UJ92ri0G4gdgok4"
local response, err = http.post(
    "http://192.168.0.52:8000/alert",
    "Alarm Triggered: Intruder",
    {
        ["API_KEY"] = API_KEY,
        ["Content-Type"] = "text/plain"
    }
)

if response then
    print("Success")
    print("Status: " .. response.getResponseCode())
    response.close()
else
    print("Failed")
    print("Error: " .. tostring(err))
end
Press Ctrl or click here to access menu Ln 19
```

Figure 2 Minecraft "pingRpi.lua"

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

1 to 255 characters and case sensitive.

Table settings

☒ Default settings

The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.

☐ Customize settings

Use these advanced features to make DynamoDB work better for your needs.

Figure 3 "alertLogs" table created

```
import secrets

MINECRAFT_API_KEY = secrets.token_urlsafe(32)

print(MINECRAFT_API_KEY)
```

Figure 4 Generate minecraft API key

```
pi@raspberrypi:~/minecraftIntrusionIoT $ cat Makefile
obj-m += intrusionMonitor.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/ build M=$(PWD) clean
```

Figure 5 "Makefile" contents

```
pi@raspberrypi:~/minecraftIntrusionIoT $ pi@raspberrypi:~/minecraftIntrusionIoT $ make
make -C /lib/modules/5.4.51+/build M=/home/pi/minecraftIntrusionIoT modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.51+'
  CC [M]  /home/pi/minecraftIntrusionIoT/intrusionMonitor.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/pi/minecraftIntrusionIoT/intrusionMonitor.mod.o
  LD [M]  /home/pi/minecraftIntrusionIoT/intrusionMonitor.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.51+'
pi@raspberrypi:~/minecraftIntrusionIoT $ ls
alertServer.py  database.py.save  intrusionMonitor.mod  main.py  Module.symvers
config.py       flipLed.py       intrusionMonitor.mod.c  main.py.save  notifications.py
config.py.save  intrusionMonitor.c  intrusionMonitor.mod.o  Makefile  __pycache__
database.py     intrusionMonitor.ko  intrusionMonitor.o  modules.order
```

Figure 6 "intrusionMonitor" LKM files

```

pi@raspberrypi:~/minecraftIntrusionIoT $ sudo mknod /dev/alertCounter c 20
mknod: missing operand after '20'
Try 'mknod --help' for more information.
pi@raspberrypi:~/minecraftIntrusionIoT $ sudo mknod /dev/alertCounter c 20 0
pi@raspberrypi:~/minecraftIntrusionIoT $ sudo chmod 660 /dev/alertCounter
pi@raspberrypi:~/minecraftIntrusionIoT $ sudo chown pi:pi /dev/alertCounter
pi@raspberrypi:~/minecraftIntrusionIoT $ echo "1" > /dev/alertCounter
pi@raspberrypi:~/minecraftIntrusionIoT $ cat /dev/alertCounter
1
pi@raspberrypi:~/minecraftIntrusionIoT $

```

Figure 7 Creating alertCounter LKM service

```

MINECRAFT_API_KEY = 
AWS_REGION = "us-east-1"
AWS_ACCESS_KEY = "
AWS_SECRET_KEY = "
AWS_SESSION_TOKEN = "
discordURL = "

```

Figure 8 ".env" environment file

Minecraft Intrusion Detection System

Security Dashboard

Recent Alerts Refresh

Alarm Triggered: Intruder 13/12/2025, 21:17:23
Alarm Triggered: Intruder 13/12/2025, 21:17:20
Alarm Triggered: Intruder 13/12/2025, 21:17:17
Alarm Triggered: Intruder 13/12/2025, 21:17:13
Alarm Triggered: Intruder 13/12/2025, 19:44:25
Alarm Triggered: Intruder 13/12/2025, 19:38:01
Alarm Triggered: Intruder 10/12/2025, 04:17:44
Alarm Triggered: Intruder

Figure 9 IDS web application dash board